

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Concept Mapping
Weightage:	20% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)		
Student Name:	Y. Thanushma	Reg. No.:	192425365
Section / Batch:	slot A / Hostel / 2024	Date:	

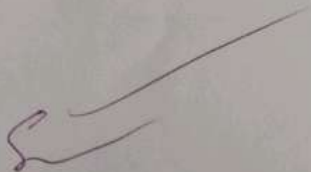
Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Draw a clear and neat concept map for each question.
- Identify all key concepts and arrange them in a logical hierarchy.
- Show meaningful linkages between concepts using arrows and connecting words.
- Compare related concepts wherever required and justify the relationships clearly.
- Ensure the concept map is well-organized, readable, and properly labelled.

Question Type	Focus Area	Questions	Marks
Concept Mapping	Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – CONCEPT MAPPING

- 1) Draw the concept map of explaining Selection sort and bubble sort
- 2) Draw the concept map of explaining Master Theorem



Code of Conduct

I Y. Thanushma / 19140530 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y. Thanushma

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Marks Evaluation:

Question No.	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SELECTION SORT

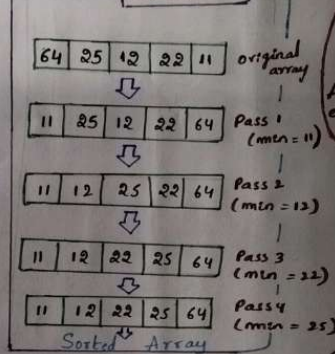
Idea

Repeatedly select the minimum element from the unsorted part and put it at the beginning.

How it works

1. Start from the first element
2. Find the minimum element in the unsorted part
3. Swap it with the first unsorted element.
4. Move the boundary one step forward.
5. Repeat the entire array sorted.

Example



Sorting Algorithms
Arrange elements in order
(A/D or D/A)

BUBBLE SORT

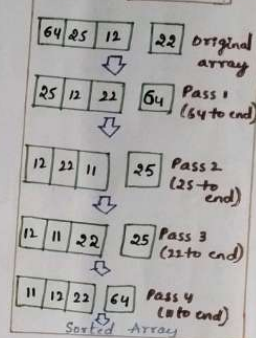
Idea

Repeatedly compare adjacent elements and swap them if they are in wrong order.

How it works

1. Start from first element
2. Compare with next element
3. Swap if they are in wrong order
4. Move to the next pair.
5. After each pass, the largest number bubbles up to end.
6. Repeat until no swaps are needed.

Example



Characteristics

- * In-Place Sorting
- * Not Stable (in general)
- * Not Adaptive
- * Simple and easy to implement.

Pros

Best Case : $O(n^2)$
Average Case : $O(n^2)$
Worst Case : $O(n^2)$
Space : $O(1)$

Pros

Simple logic.
Works well for small datasets.
less no. of swaps ($n-1$ swaps)

Cons

Always $O(n^2)$ time
Not efficient for large datasets

COMPARISON OVERVIEW

Feature	Selection Sort	Bubble Sort
Strategy	Select min & place	Swap adjacent elements
Swaps	Few ($n-1$)	More (many times)
Best case	$O(n^2)$	$O(n)$
Stability	Not	Yes
Adaptive	No	Yes

Pros

Simple to implement
Stable
Adaptive (fast for sorted data)

Cons

Many swaps
Inefficient for large data sets

Characteristics

- * In place sorting
- * Stable
- * Adaptive (best case $O(n)$)
- * Easy to understand

Complexity

Best case : $O(n)$
Average case : $O(n^2)$
Worst case : $O(n^2)$
Space : $O(1)$

MASTER THEOREM

MASTER THEOREM

Solves recurrences of the form

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where,

$a \geq 1$: number of subproblems
 $b > 1$: factor by which problem is divided
 $f(n)$: time to divide and combine sub
 $T(n)$: Time complexity of algorithm

EXAMPLES

Recurrence $T(n)$	a	b	$f(n)$	Compare $f(n)$ with $n \log_b a$	Case	Result $T(n)$
$T(n) = 2T\left(\frac{n}{2}\right) + n$	2	2	n	$n' = n$ (same order)	Case 2 ($k=0$)	$\Theta(n \log n)$
$T(n) = 2T\left(\frac{n}{2}\right) + 1$	2	2	1	$1 = O(n^{1-\epsilon})$ (smaller)	Case 1	$\Theta(n)$
$T(n) = 2T\left(\frac{n}{2}\right) + n^2$	2	2	n^2	$n^2 = \Omega(n^{1+\epsilon})$ (larger)	Case 3	$\Theta(n^2)$
$T(n) = 3T\left(\frac{n}{2}\right) + n \log n$	3	2	$n \log n$	$n \log n = O(n^{\log_2 3})$ (smaller)	Case 1	$\Theta(n^{\log_2 3})$

Regularity Condition (For case 3)

There must exist a constant $c < 1$ such that

$$af\left(\frac{n}{b}\right) \leq cf(n) \text{ for sufficiently large } n$$

This condition prevents the work of sub-problems from dominating $f(n)$ too much

How to Apply

1. Identify a, b and $f(n)$
2. Compare $n \log_b a$
3. Compare $f(n)$ with $n \log_b a$
4. Apply the corresponding case (1, 2, or 3) to get $T(n)$

MASTER THEOREM

A Powerful tool to solve Divide and conquer

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Intuition

- Case 1 : $f(n)$ is small $\rightarrow$ Subproblem work dominates
- Case 2 : $f(n)$ is balanced $\rightarrow$ Both parts contribute
- Case 3 : $f(n)$ is large $\rightarrow$ combine work dominates

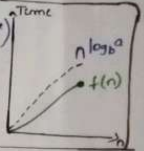
The Three Cases

Case 1

$$\text{If } f(n) = O(n^{\log_b a - \epsilon}) \text{ for some } \epsilon > 0$$

$$T(n) = \Theta(n^{\log_b a})$$

$f(n)$ grows polynomially slower

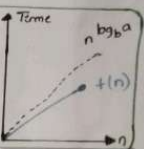


Case 2

$$\text{If } f(n) = \Theta(n^{\log_b a} \log^k n) \text{ for } k \geq 0$$

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

$f(n)$ is same order

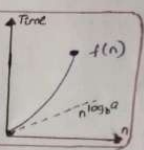


Case 3

$$\text{If } f(n) = \Omega(n^{\log_b a + \epsilon}) \text{ for some } \epsilon > 0$$

$$T(n) = \Theta(f(n))$$

$f(n)$ grows polynomially faster



Where is it Used

Analyzing Divide & Conquer algorithms
 (eg: Merge sort, Quick sort, Binary search)
 Design and analysis of efficient algorithms
 Compare growth rates of recursive algorithms
 Helps avoid solving recurrences manually

Key Takeaway

Master Theorem gives the time complexity in one step by comparing $f(n)$ with $n \log_b a$ using the three cases.